SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Application Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**M Venkata Sai**	**Reg. No.**	**192425223**
Section/Batch	**Slot D / 2024**	**Date**	**20-10-2026**

Code of Conduct:

I, M Venkata Sai (192425223) , certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M Sai

APPLICATION-BASED QUESTION

Assignment – Question 20

Question: A website loads slowly despite a fast Internet connection. Apply your knowledge of HTTP requests, server processing, client-server communication, and resource delivery to identify possible causes and recommend improvements.

1. Introduction

A website is a collection of web pages, images, videos, scripts, and other resources that users access through a web browser. Normally, a website should load quickly when the user has a good Internet connection. However, sometimes a website loads slowly even when the Internet speed is high.

This problem may occur due to several reasons, such as too many HTTP requests, slow server processing, inefficient database queries, large resource files, and delays in client-server communication.

Website performance depends not only on Internet speed but also on how efficiently the server processes requests and delivers resources to the browser. Therefore, identifying the main cause of slow loading is important for improving the performance of a website.

- **2. Objectives**

The main objectives of this analysis are:

1. To understand the reasons for slow website loading.
2. To study the role of HTTP requests and responses.
3. To identify server-side processing problems.
4. To understand delays in client-server communication.
5. To examine the effect of images, scripts, and other resources.
6. To recommend suitable methods to improve website performance.

- **3. Working of a Website**

A website generally follows a client-server architecture. The client is the user's web browser, and the server stores or generates the content required by the website.

The website loading process consists of the following steps:

Step 1: Entering the URL

The user enters a website address in the browser, such as www.example.com.

Step 2: DNS Lookup

The browser uses the Domain Name System (DNS) to find the IP address associated with the domain name.

Step 3: Establishing a Connection

The browser establishes a connection with the server. Depending on the protocol, this may involve a TCP connection and TLS encryption setup, or a QUIC connection for HTTP/3.

Step 4: Sending an HTTP Request

The browser sends an HTTP request to ask the server for the required webpage or resource.

Step 5: Server Processing

The server processes the request, executes the necessary application logic, and retrieves information from a database if required.

Step 6: Sending an HTTP Response

The server returns an HTTP response containing the requested content and a status code, such as 200 OK when the request succeeds.

Step 7: Resource Download and Rendering

The browser downloads resources such as CSS, JavaScript, and images, processes them, and displays the webpage.

If any of these steps takes too much time, the website may load slowly even when the Internet connection is fast.

- **4. Possible Causes of Slow Website Loading**

4.1 Too Many HTTP Requests

HTTP is a protocol used for communication between a web browser and a web server. A webpage may require multiple requests to download HTML documents, CSS stylesheets, JavaScript files, images, fonts, and other resources.

When a website contains too many separate resources, the browser must manage many requests. Some resources may also depend on others before they can be downloaded or displayed.

Example: A webpage containing 100 images and several CSS and JavaScript files may take longer to display than a webpage with fewer, optimized resources.

Solution:

- Remove unnecessary resources.
- Combine files when appropriate.
- Use image sprites where suitable.
- Use modern HTTP protocols such as HTTP/2 or HTTP/3.
- Avoid loading resources that are not required immediately.

4.2 Slow Server Processing

The web server receives requests and generates responses. If the server has insufficient memory, CPU capacity, or processing resources, it may take longer to respond.

High traffic can also overload the server. When many users access a website simultaneously, requests may wait in a queue before being processed.

Example: An online shopping website may respond slowly during a major sale because thousands of users are accessing it at the same time.

Solution:

- Increase server capacity when necessary.
- Optimize application code.
- Use load balancing to distribute requests across multiple servers.

- Monitor server CPU, memory, and response times.
- Scale server resources according to traffic.

4.3 Client-Server Communication Delays

Client-server communication involves exchanging requests and responses between the browser and the server. Even with a fast Internet connection, delays can occur because of network latency, distant servers, DNS resolution, or connection setup.

Latency is the time taken for data to travel between two points. A connection may have high bandwidth but still experience significant latency.

Example: If a website's server is located in another country, the browser may experience additional delays while communicating with it.

Solution:

- Host the website closer to its main audience where practical.
- Use a Content Delivery Network (CDN).
- Reduce unnecessary redirects.
- Reuse connections where possible.
- Optimize DNS configuration and connection setup.

4.4 Large Images and Media Files

Images, videos, and other media files may consume a large amount of data. If these files are not optimized, they take longer to download and may delay the display of important page content.

For example, uploading a high-resolution image of several megabytes when a much smaller image would be sufficient can unnecessarily increase loading time.

Solution:

- Compress images before uploading them.
- Use modern image formats such as WebP or AVIF where suitable.
- Resize images to match their display dimensions.
- Use responsive images for different screen sizes.
- Enable lazy loading for images that are below the visible area.
- Stream video rather than requiring large files to download completely.

4.5 Inefficient JavaScript and CSS

JavaScript adds interactive features to a website, while CSS controls its appearance. Large or poorly optimized scripts can delay page rendering and make the browser slow to respond.

Some JavaScript files may block the browser from processing other important resources. Large stylesheets may also delay the display of content.

Solution:

- Remove unused JavaScript and CSS.
- Minify files by removing unnecessary characters.
- Use defer or async for suitable JavaScript files.

- Load critical CSS efficiently.
- Split large JavaScript bundles into smaller parts when appropriate.

4.6 Slow Database Queries

Dynamic websites frequently retrieve information from databases. If database queries are inefficient, the server may take a long time to generate a response.

Missing indexes, poorly designed queries, excessive database requests, and large result sets can increase processing time.

Example: A college website may load slowly when it retrieves student details, examination results, and attendance information using inefficient database queries.

Solution:

- Optimize SQL queries.
- Create appropriate database indexes.
- Retrieve only the required data.
- Avoid repeated queries for the same information.
- Use caching for frequently accessed data.
- Monitor database response times.

4.7 Lack of Caching

Caching means temporarily storing copies of frequently used resources so that they can be accessed more quickly later.

Without caching, the browser may repeatedly download the same images, stylesheets, and scripts. Similarly, the server may repeatedly generate the same responses.

Solution:

- Enable browser caching for static resources.
- Use appropriate HTTP cache headers.
- Introduce server-side caching for frequently requested data.
- Use a CDN to cache suitable resources closer to users.
- Configure cache expiration and validation correctly.

Caching can reduce unnecessary requests, server workload, and loading time.

- **5. Role of HTTP Requests and Responses**

HTTP stands for Hypertext Transfer Protocol. It is used for communication between web clients and servers. An HTTP request contains information such as the request method, resource path, headers, and sometimes a request body. The server processes the request and returns a response containing a status code, headers, and possibly a response body.

Common HTTP methods include:

- **GET:** Retrieves a resource from the server.
- **POST:** Sends data to the server.
- **PUT:** Creates or replaces a resource, depending on the application.

- **DELETE:** Requests deletion of a resource.

Common HTTP response status codes include:

- **200 OK:** The request was successful.
- **301 Moved Permanently:** The resource has a permanent new location.
- **404 Not Found:** The requested resource could not be found.
- **500 Internal Server Error:** The server encountered an unexpected error.

Repeated redirects, unnecessary requests, slow responses, and failed resource downloads can all affect website performance.

To improve performance, developers should reduce unnecessary requests, correct broken resource links, avoid redirect chains, and monitor server response times.

- **6. Recommended Improvements**

6.1 Optimize HTTP Requests

Developers should reduce unnecessary requests and ensure that essential resources load efficiently. Modern protocols such as HTTP/2 and HTTP/3 support more efficient communication, although they cannot eliminate delays caused by slow application processing.

6.2 Improve Server Performance

The server should have sufficient processing power, memory, and network capacity. Developers should optimize the application code and use load balancing or horizontal scaling when required.

6.3 Compress Resources

Gzip or Brotli compression can reduce the size of suitable text-based files, such as HTML, CSS, JavaScript, and JSON. Smaller files require less data to be transferred over the network.

6.4 Use a Content Delivery Network

A CDN stores copies of suitable website resources on servers distributed across different locations. When a user requests a resource, it can be delivered from a nearby CDN server instead of the original server.

This reduces delivery distance and can improve loading speed.

6.5 Enable Browser and Server Caching

Browser caching allows static resources to be reused across visits. Server-side caching can reduce repeated processing and database queries.

Cache settings should be configured carefully so that users do not receive outdated content when updates are required.

6.6 Optimize Database Operations

Database queries should be reviewed for unnecessary joins, repeated requests, and missing indexes. Frequently requested data can be cached when appropriate.

6.7 Monitor Website Performance

Developers should use browser developer tools, server logs, and performance monitoring tools to identify the exact cause of delays.

Useful measurements include:

- **Time to First Byte (TTFB):** Time until the browser receives the first byte of the response.
- **Largest Contentful Paint (LCP):** Time taken for the main visible content to appear.
- **Total page size:** Amount of data required to load the page.
- **Number of requests:** Number of resources requested by the browser.
- **Server response time:** Time taken by the server to respond to a request.

These measurements help developers identify whether the problem is caused by the network, server, database, or browser rendering.

- **7. Example Scenario**

Consider an online shopping website that loads slowly even though the user has a fast Internet connection. The developer investigates the website and identifies the following problems:

1. The homepage contains many large product images.
2. Several JavaScript files delay page rendering.
3. Database queries take too long to retrieve product details.
4. The server experiences high traffic during peak hours.
5. Static resources are not cached properly.

The developer applies the following improvements:

1. Compresses product images and uses modern image formats.
2. Removes unused JavaScript and delays nonessential scripts.
3. Optimizes database queries and introduces suitable indexes.
4. Uses load balancing to distribute incoming requests.
5. Enables browser caching and uses a CDN.

After these changes, the website can deliver resources more efficiently, reduce server workload, and display important content sooner. The actual improvement depends on which problems were responsible for the original delay.

- **8. Expected Results**

By applying the recommended improvements, the website can achieve the following results:

- Faster loading of webpages.
- Reduced HTTP request overhead.
- Lower server processing time.
- Improved database performance.
- Reduced network data transfer.
- Better use of browser caching.
- Faster delivery of images and other resources.
- Improved user satisfaction and website reliability.

- **9. Conclusion**

A website may load slowly despite a fast Internet connection because website performance depends on several factors beyond bandwidth. These include HTTP request handling, server processing, client-server communication, database efficiency, resource size, and caching.

The problem can be addressed by optimizing HTTP requests, improving server capacity, compressing resources, reducing database delays, using a CDN, and enabling suitable caching mechanisms. Performance monitoring helps developers identify the main bottlenecks and choose the correct solutions.

Therefore, a combination of efficient client-server communication, optimized resource delivery, and effective server management is essential for achieving faster website loading and a better user experience.

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimize code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, wellstructured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lack's structure and proper documentation	Poorly written code with no documentation